

20200519 async await

1. 问题1

```
async function t1() {  
  let a = await "lagou";  
  console.log(a);  
}  
t1()
```

问题解析

await是一个表达式，如果后面不是一个promise对象，就直接返回对应的值。

所以问题1可以理解为

```
async function t1() {  
  let a = "lagou";  
  console.log(a); // lagou  
}  
t1()
```

2. 问题2

```
async function t2() {  
  let a = await new Promise((resolve) => {});  
  console.log(a);  
}  
t2()
```

问题解析

await后面如果跟一个promise对象，await将等待这个promise对象的resolve状态的值value，且将这个值返回给前面的变量，此时的promise对象的状态是一个pending状态，没有resolve状态值，所以什么也打印不了。

3. 问题3

```
async function t3() {  
  let a = await new Promise((resolve) => {  
    resolve();  
  });  
  console.log(a); // undefined  
}  
t3()
```

await后面如果跟一个promise对象，await将等待这个promise对象的resolve状态的值value，且将这个值返回给前面的变量，此时的promise对象的状态是一个resolve状态，但是它的状态值是undefined，所以打印出undefined。

4. 问题4

```
async function t4() {  
  let a = await new Promise((resolve) => {  
    resolve("hello");  
  });  
  console.log(a); // hello  
}  
t4()
```

await后面如果跟一个promise对象，await将等待这个promise对象的resolve状态的值，且将这个值返回给前面的变量，此时的promise对象的状态是一个resolve状态，它的状态值是hello，所以打印出hello。

5. 问题5

```
async function t5() {
  let a = await new Promise((resolve) => {
    resolve("hello");
  }).then(() => {
    return "lala";
  });
  console.log(a); //lala
}
t5()
```

await后面如果跟一个promise对象，await将等待这个promise对象的resolve状态的值，且将这个值返回给前面的变量，此时的promise对象的状态是一个resolve状态，它的状态值是hello，紧接着后面又执行了一个then方法，then方法又会返回一个全新的promise对象，且这个then方法中的返回值会作为这个全新的promise中resolve的值，所以最终的结果是lala。

6. 问题6

```
async function t6() {
  let a = await fn().then((res)=>{return res})
  console.log(a); //undefined
}
async function fn(){
  await new Promise((resolve)=>{
    resolve("lagou")
  })
}
t6()
```

问题解析

async函数执行返回一个promise对象，且async函数内部的返回值会当作这个promise对象resolve状态的值

```
async function fn() {
  return "la";
}
var p = fn();
console.log(p); //Promise {<resolved>: "la"}
               //__proto__: Promise
               //[[PromiseStatus]]: "resolved"
               //[[PromiseValue]]: "la"
```

首先考虑 fn() 执行返回一个promise对象，因为fn执行没有返回值，所以这个promise对象的状态resolve的值是undefined，且将这个undefined当作下一个then中回调函数的参数，所以打印的结果是undefined

7. 问题7

```
async function t7() {
  let a = await fn().then((res)=>{return res})
  console.log(a);
}
async function fn(){
  await new Promise((resolve)=>{
    resolve("lagou")
  })
  return "lala"
}
t7()
```

首先考虑 `fn()` 执行返回一个promise对象，因为`fn()`执行有返回值lala，所以这个promise对象的状态resolve的值是lala，且将这个lala当作下一个then中回调函数的参数，所以打印的结果是lala。

注意细节

- `async`函数执行的返回结果是一个promise对象，这个函数的返回值是这个promise状态值resolve的值
- `await`后面如果不是一个promise对象，将直接返回这个值
- `await`后面如果是一个promise对象，将会把这个promise的状态resolve的值返回出去。

以上没有考虑reject状态。